

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ

з лабораторної роботи №3

з дисципліни "Основи Програмної Інженерії"

Виконала

ст. гр. ПЗПІ-25-3

Головко Є.А.

Прийняли:

Каук В. І.

Гребенюк В. О.

Харків 2026

1. Тема та мета лабораторної роботи.

Тема: Модульне тестування програмного коду (Unit Testing).

Мета роботи: Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

2. Вихідний код реалізованого модуля з коментарями.

```
using System;

namespace ImpulseApp
{
    public class BiometricsCalculator
    {
        // Метод 1: Розрахунок Індексу Маси Тіла (ІМТ)
        public double CalculateBMI(double weightKg, double
heightCm)
        {
            if (weightKg <= 0 || heightCm <= 0)
                throw new ArgumentException("Вага та зріст повинні
бути більшими за нуль.");

            double heightM = heightCm / 100.0;
            double bmi = weightKg / (heightM * heightM);
            return Math.Round(bmi, 2);
        }

        // Метод 2: Розрахунок добової норми калорій (Формула
Харріса-Бенедикта)
        public double CalculateDailyCalories(double weightKg,
double heightCm, int age, string gender)
        {
            if (weightKg <= 0 || heightCm <= 0 || age <= 0)
                throw new ArgumentException("Показники повинні
бути додатними числами.");

            double bmr;
```

```

        if (gender.ToLower() == "male" || gender.ToLower() ==
"чоловік")
        {
            bmr = 88.36 + (13.4 * weightKg) + (4.8 * heightCm)
- (5.7 * age);
        }
        else if (gender.ToLower() == "female" ||
gender.ToLower() == "жінка")
        {
            bmr = 447.6 + (9.2 * weightKg) + (3.1 * heightCm)
- (4.3 * age);
        }
        else
        {
            throw new ArgumentException("Невідома статъ.
Вкажіть 'чоловік' або 'жінка'.");
        }
        return Math.Round(bmr, 2);
    }

    // Метод 3: Розрахунок норми води в день залежно від ваги
та часу тренування
    public double CalculateWaterNorm(double weightKg, int
workoutMinutes)
    {
        if (weightKg <= 0 || workoutMinutes < 0)
            throw new ArgumentException("Вага має бути > 0, а
час тренування не може бути від'ємним.");

        // Базова норма: 30 мл на 1 кг ваги
        double baseWaterLiters = (weightKg * 30) / 1000.0;

        // Додаткова вода: 500 мл (0.5 л) за кожні 30 хвилин
тренування
        double extraWaterLiters = (workoutMinutes / 30.0) *
0.5;

        return Math.Round(baseWaterLiters + extraWaterLiters,
2);
    }
}

```

3. Таблиця проєктування тестів: «Тест-кейс | Вхідні дані | Очікуваний результат | Техніка (EP/BVA) | Статус (pass/fail)».

Тест-кейс	Вхідні дані	Очікуваний	Техніка	Статус
-----------	-------------	------------	---------	--------

		результат	(EP/BVA)	(pass/fail)
ТС-01 (BMI: Нормальні дані)	Вага: 70, Зріст: 175	22.86	EP (позитивний)	pass
ТС-02 (BMI: Межа, вага = 0)	Вага: 0, Зріст: 175	ArgumentException (помилка)	BVA (негативний)	pass
ТС-03 (BMI: Межа, зріст = 0)	Вага: 70, Зріст: 0	ArgumentException (помилка)	BVA (негативний)	pass
ТС-04 (BMI: Від'ємна вага)	Вага: -5, Зріст: 175	ArgumentException (помилка)	BVA (негативний)	pass
ТС-05 (BMI: Від'ємний зріст)	Вага: 70, Зріст: -175	ArgumentException (помилка)	BVA (негативний)	pass
ТС-06 (Калорії: Чоловік)	Вага: 75, Зріст: 180, Вік: 25, Стать: "чоловік"	1814.86	EP (позитивний)	pass
ТС-07 (Калорії: Жінка)	Вага: 60, Зріст: 165, Вік: 30, Стать: "жінка"	1382.10	EP (позитивний)	pass
ТС-08 (Калорії: Невідома стать)	Вага: 70, Зріст: 170, Вік: 25, Стать: "кіт"	ArgumentException (помилка)	BVA (негативний)	pass
ТС-09 (Калорії: Нульовий вік)	Вага: 70, Зріст: 170, Вік: 0, Стать: "жінка"	ArgumentException (помилка)	BVA (негативний)	pass
ТС-10	Вага: 0,	ArgumentExc	BVA	pass

(Калорії: Нульова вага)	Зріст: 170, Вік: 25, Стать: "чоловік"	eption (помилка)	(негативний)	
ТС-11 (Вода: Без тренування)	Вага: 60, Тренування: 0 хв	1.80 (літра)	EP (позитивний)	pass
ТС-12 (Вода: Тренування 30 хв)	Вага: 60, Тренування: 30 хв	2.30 (літра)	EP (позитивний)	pass
ТС-13 (Вода: Тренування 45 хв)	Вага: 60, Тренування: 45 хв	2.55 (літра)	EP (позитивний)	pass
ТС-14 (Вода: Нульова вага)	Вага: 0, Тренування: 30 хв	ArgumentExc eption (помилка)	BVA (негативний)	pass
ТС-15 (Вода: Від'ємний час)	Вага: 60, Тренування: -15 хв	ArgumentExc eption (помилка)	BVA (негативний)	pass

4. Вихідний код тестового набору з коментарями.

```
using System;
using Xunit;

namespace ImpulseApp.Tests
{
    public class BiometricsCalculatorTests
    {
        // Тести для методу: CalculateBMI

        [Fact]
        public void CalculateBMI_NormalData_ReturnsCorrectBMI()
        {
            // Техніка: EP (позитивний) - ТС-01
            // Arrange (Підготовка)
            var calculator = new BiometricsCalculator();
            double weight = 70;
            double height = 175;

            // Act (Виконання)
```

```

        double result = calculator.CalculateBMI(weight,
height);

        // Assert (Перевірка)
        Assert.Equal(22.86, result);
    }

    [Fact]
    public void
CalculateBMI_ZeroWeight_ThrowsArgumentException()
    {
        // Техніка: BVA (негативний) - TC-02
        // Arrange
        var calculator = new BiometricsCalculator();
        double weight = 0; // Граничне значення
        double height = 175;

        // Act & Assert (Для винятків дія і перевірка
об'єднуються)
        Assert.Throws<ArgumentException>(() =>
calculator.CalculateBMI(weight, height));
    }

    [Fact]
    public void
CalculateBMI_ZeroHeight_ThrowsArgumentException()
    {
        // Техніка: BVA (негативний) - TC-03
        // Arrange
        var calculator = new BiometricsCalculator();
        double weight = 70;
        double height = 0; // Граничне значення

        // Act & Assert
        Assert.Throws<ArgumentException>(() =>
calculator.CalculateBMI(weight, height));
    }

    [Fact]
    public void
CalculateBMI_NegativeWeight_ThrowsArgumentException()
    {
        // Техніка: EP (негативний) - TC-04
        // Arrange
        var calculator = new BiometricsCalculator();
        double weight = -5; // Недопустимий клас
        double height = 175;

```

```

        // Act & Assert
        Assert.Throws<ArgumentException>(() =>
calculator.CalculateBMI(weight, height));
    }

    [Fact]
    public void
CalculateBMI_NegativeHeight_ThrowsArgumentException()
    {
        // Техніка: ЕР (негативний) - ТС-05
        // Arrange
        var calculator = new BiometricsCalculator();
        double weight = 70;
        double height = -175; // Недопустимий клас

        // Act & Assert
        Assert.Throws<ArgumentException>(() =>
calculator.CalculateBMI(weight, height));
    }

    // Тести для методу: CalculateDailyCalories

    [Fact]
    public void
CalculateDailyCalories_Male_ReturnsCorrectCalories()
    {
        // Техніка: ЕР (позитивний) - ТС-06
        // Arrange
        var calculator = new BiometricsCalculator();
        double weight = 75;
        double height = 180;
        int age = 25;
        string gender = "чоловік";

        // Act
        double result =
calculator.CalculateDailyCalories(weight, height, age, gender);

        // Assert
        Assert.Equal(1814.86, result);
    }

    [Fact]
    public void
CalculateDailyCalories_Female_ReturnsCorrectCalories()
    {
        // Техніка: ЕР (позитивний) - ТС-07
        // Arrange

```

```

        var calculator = new BiometricsCalculator();
        double weight = 60;
        double height = 165;
        int age = 30;
        string gender = "жінка";

        // Act
        double result =
calculator.CalculateDailyCalories(weight, height, age, gender);

        // Assert
        Assert.Equal(1382.10, result);
    }

    [Fact]
    public void
CalculateDailyCalories_UnknownGender_ThrowsArgumentException()
    {
        // Техніка: ЕР (негативний) - TC-08
        // Arrange
        var calculator = new BiometricsCalculator();
        string gender = "кіт"; // Недопустимий клас
еквівалентності

        // Act & Assert
        Assert.Throws<ArgumentException>(() =>
calculator.CalculateDailyCalories(70, 170, 25, gender));
    }

    [Fact]
    public void
CalculateDailyCalories_ZeroAge_ThrowsArgumentException()
    {
        // Техніка: BVA (негативний) - TC-09
        // Arrange
        var calculator = new BiometricsCalculator();
        int age = 0; // Граничне значення

        // Act & Assert
        Assert.Throws<ArgumentException>(() =>
calculator.CalculateDailyCalories(70, 170, age, "жінка"));
    }

    [Fact]
    public void
CalculateDailyCalories_ZeroWeight_ThrowsArgumentException()
    {
        // Техніка: BVA (негативний) - TC-10

```

```

        // Arrange
        var calculator = new BiometricsCalculator();
        double weight = 0; // Граничне значення

        // Act & Assert
        Assert.Throws<ArgumentException>(() =>
calculator.CalculateDailyCalories(weight, 170, 25, "чоловік"));
    }

    // Тести для методу: CalculateWaterNorm

    [Fact]
    public void CalculateWaterNorm_NoWorkout_ReturnsBaseNorm()
    {
        // Техніка: BVA (позитивний) - TC-11
        // Arrange
        var calculator = new BiometricsCalculator();
        double weight = 60;
        int workoutMinutes = 0; // Нижня межа допустимих
значень

        // Act
        double result = calculator.CalculateWaterNorm(weight,
workoutMinutes);

        // Assert
        Assert.Equal(1.80, result);
    }

    [Fact]
    public void
CalculateWaterNorm_30MinWorkout_ReturnsCorrectNorm()
    {
        // Техніка: EP (позитивний) - TC-12
        // Arrange
        var calculator = new BiometricsCalculator();
        double weight = 60;
        int workoutMinutes = 30;

        // Act
        double result = calculator.CalculateWaterNorm(weight,
workoutMinutes);

        // Assert
        Assert.Equal(2.30, result);
    }

    [Fact]

```

```

        public void
CalculateWaterNorm_45MinWorkout_ReturnsCorrectNorm()
    {
        // Техніка: ЕР (позитивний) - ТС-13
        // Arrange
        var calculator = new BiometricsCalculator();
        double weight = 60;
        int workoutMinutes = 45;

        // Act
        double result = calculator.CalculateWaterNorm(weight,
workoutMinutes);

        // Assert
        Assert.Equal(2.55, result);
    }

    [Fact]
    public void
CalculateWaterNorm_ZeroWeight_ThrowsArgumentException()
    {
        // Техніка: BVA (негативний) - ТС-14
        // Arrange
        var calculator = new BiometricsCalculator();
        double weight = 0; // Граничне значення

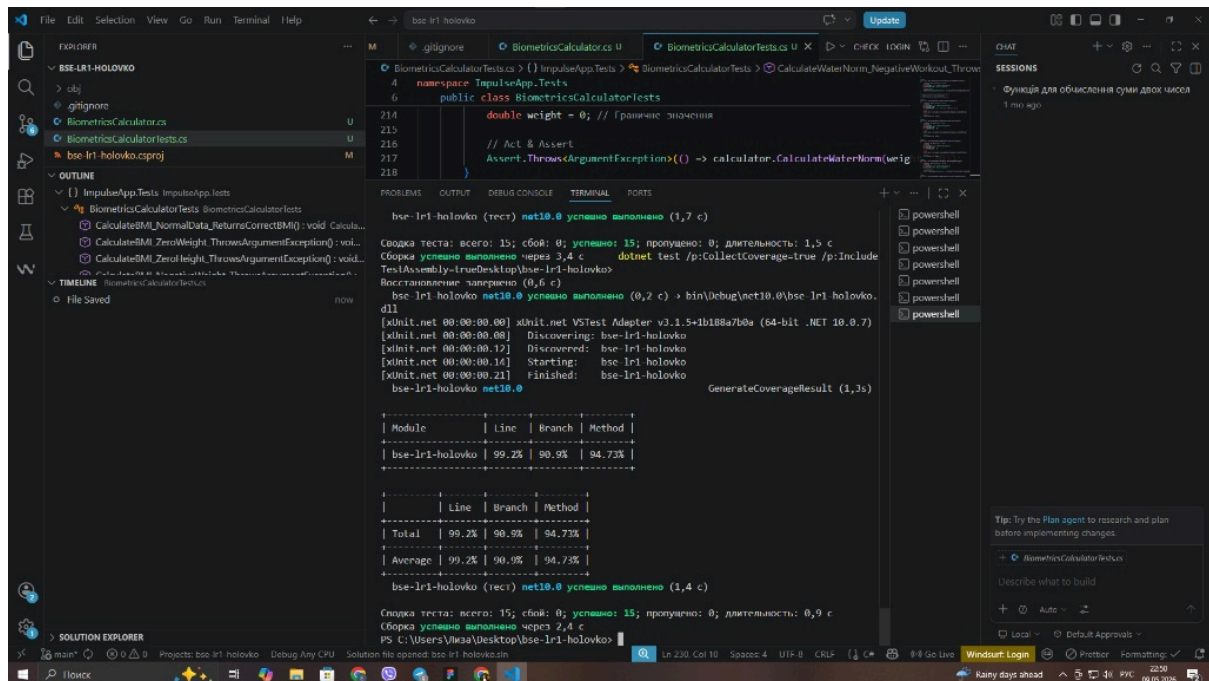
        // Act & Assert
        Assert.Throws<ArgumentException>(() =>
calculator.CalculateWaterNorm(weight, 30));
    }

    [Fact]
    public void
CalculateWaterNorm_NegativeWorkout_ThrowsArgumentException()
    {
        // Техніка: ЕР (негативний) - ТС-15
        // Arrange
        var calculator = new BiometricsCalculator();
        int workoutMinutes = -15; // Недопустимий клас

        // Act & Assert
        Assert.Throws<ArgumentException>(() =>
calculator.CalculateWaterNorm(60, workoutMinutes));
    }
}

```

5. Скріншот або HTML-звіт покриття коду з відображенням відсотку line coverage.



6. Посилання на Git-репозиторій з кодом модуля та тестами.

<https://github.com/yelyzavetaholovko-ui/Impulse/tree/yelyzavetaholovko-ui/bse-lr1-holovko>

7. Висновки (аналіз отриманих результатів покриття, виявлені проблеми, шляхи поліпшення).

Під час виконання лабораторної роботи було розроблено модуль BiometricsCalculator для інтеграції в систему Impulse. Для перевірки його працездатності використовувалися методи еквівалентного розбиття (ЕР) та аналізу граничних значень (BVA), що забезпечило створення 15 ефективних тест-кейсів. Завдяки застосуванню фреймворку xUnit разом з інструментом Coverlet тестування було повністю автоматизоване, а покриття коду (Line coverage) досягло показника 99.2%. Це підтверджує високий рівень якості реалізації модуля та значно знижує ризик виникнення критичних помилок.